МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

Институт информационных технологий и анализа данных
Кафедра информационных технологий и систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине «Информатика и основы программирования»

Лабораторная работа 7. Обработка списков и таблиц

Студент
гр. БИН-26-03 _____ А. В. Лямин

Студент
гр. БИН-26-03 _____ А. В. Тимченко

Преподаватель _____ М. В. Водяницкий

Владивосток 2026

1 Выполнение работы

1.1 Задание 1. Корректировка остатков

В первом задании требуется выполнить корректировку списка целых чисел, задающего остатки товаров на складе. Пользователь вводит целые числа одной строкой через пробел. Программа создает независимую копию исходного списка и заменяет в ней все отрицательные значения нулями с помощью прямого присваивания по индексу. Исходный список сохраняется в неизменном виде. На экран последовательно выводятся исходный и исправленный списки остатков.

Алгоритм решения первого задания представлен на рисунке 1.

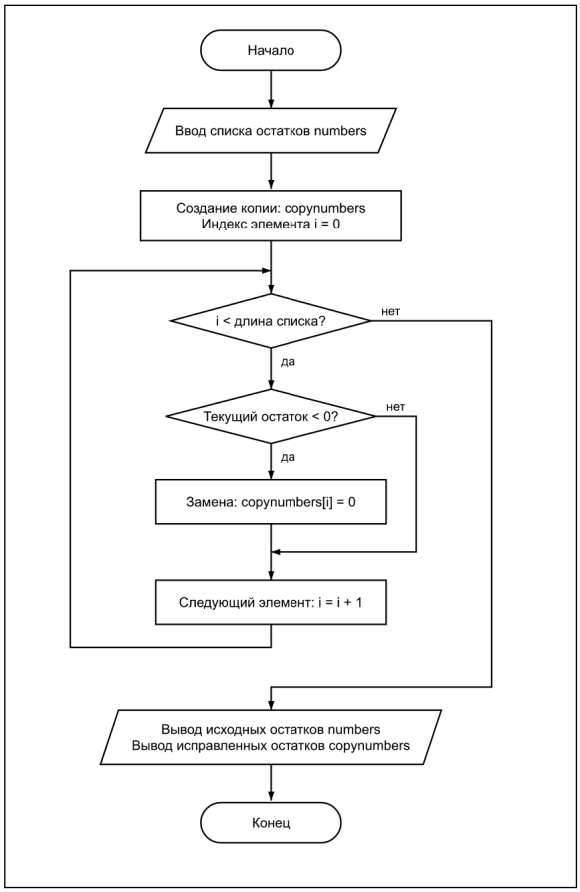


Рисунок 1 – Блок-схема программы для задания 1

На блок-схеме показаны ввод исходного списка остатков `numbers`, создание копии `copynumbers`, цикл перебора элементов по индексу i, проверка условия отрицательности текущего остатка (`copynumbers[i] < 0`), замена отрицательного значения нулем, шаг инкремента индекса, слияние альтернативных линий потока со стрелкой на стыке в соответствии с ГОСТ 19.701-90 и итоговый вывод списков исходных и исправленных остатков.

Текст программы для задания 1 представлен на рисунке 2.

```
numbers = list(map(int, input().split()))
copynumbers = numbers.copy()
for item in range(len(copynumbers)):
    if copynumbers[item] < 0:
        copynumbers[item] = 0
print(f'Исходные остатки: {numbers}')
print(f'Исправленные остатки: {copynumbers}')
```

Рисунок 2 – Текст программы для задания 1

Для сохранения исходных данных применяется метод `numbers.copy()`, создающий поверхностную копию списка, а обход элементов копии выполняется по индексам в диапазоне `range(len(copynumbers))` с перезаписью отрицательных значений нулями.

Результат работы программы при вводе «12 −3 0 7 −1» представлен на рисунке 3.

```
12 -3 0 7 -1
Исходные остатки: [12, -3, 0, 7, -1]
Исправленные остатки: [12, 0, 0, 7, 0]
```

Рисунок 3 – Результат работы программы для задания 1

1.2 Задание 2. Отбор значений

Во втором задании требуется с помощью механизма спискового включения (list comprehension) отобрать из введенной последовательности целых чисел положительные нечетные значения и сформировать список их квадратов. Порядок следования и кратность отобранных элементов сохраняются. При отсутствии подходящих чисел результатом работы является пустой список.

Алгоритм решения второго задания представлен на рисунке 4.

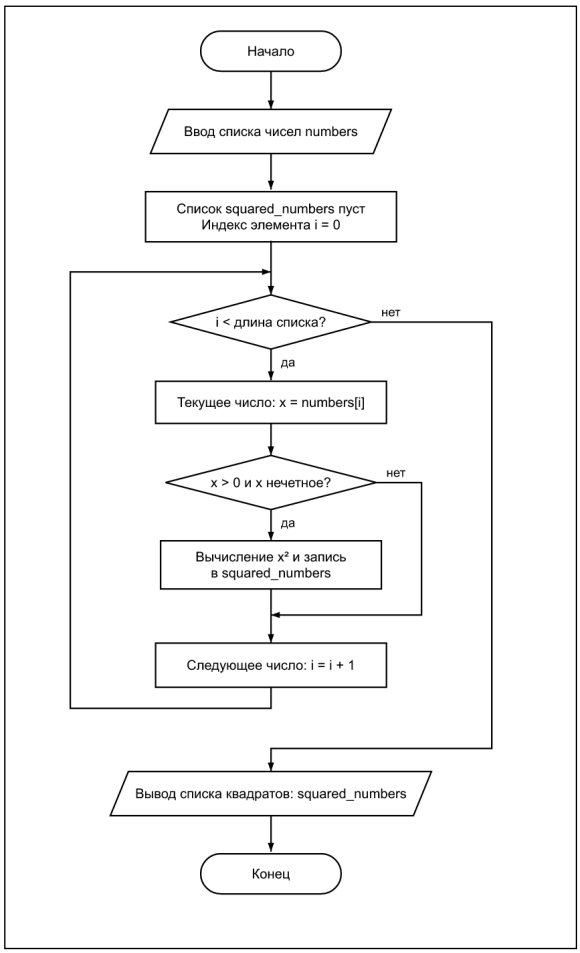


Рисунок 4 – Блок-схема программы для задания 2

На блок-схеме отражены ввод исходного списка `numbers`, инициализация результирующего списка квадратов `squared_numbers`, цикл перебора элементов по индексу i, извлечение текущего числа x, проверка составного условия (положительность и нечетность числа: $x > 0$ и x нечетное), вычисление квадрата x^2 и его запись в список результатов, переход к следующему числу, слияние линий управления и вывод сформированного списка квадратов. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 2 представлен на рисунке 5.

```
numbers = list(map(int, input().split()))
squared_numbers = [x**2 for x in numbers if x % 2 != 0 and x > 0]
print(f'Квадраты: {squared_numbers}')
```

Рисунок 5 – Текст программы для задания 2

Построение списка осуществляется компактным генераторным выражением `[x**2 for x in numbers if x % 2 != 0 and x > 0]`, объединяющим фильтрацию по условию и поэлементное возведение в квадрат.

Результат работы программы при вводе «-3 5 0 2 1 5» представлен на рисунке 6.

```
-3 5 0 2 1 5
Квадраты: [25, 1, 25]
```

Рисунок 6 – Результат работы программы для задания 2

1.3 Задание 3. Индексы совпадений

В третьем задании требуется определить позиции всех элементов списка, равных заданному эталонному значению. Пользователь вводит список целых чисел в первой строке и искомое число `total` во второй строке. Программа строит список индексов элементов, совпадающих с `total`, в порядке возрастания. Поиск позиций выполняется с использованием стандартной функции `enumerate()`, обеспечивающей нумерацию с нуля. При отсутствии совпадений выводится пустой список.

Алгоритм решения третьего задания представлен на рисунке 7.

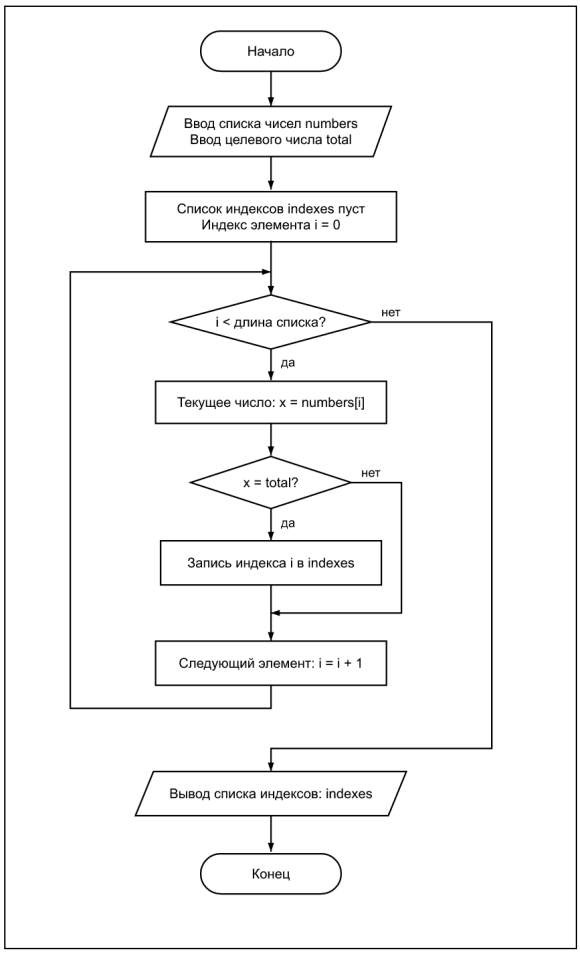


Рисунок 7 – Блок-схема программы для задания 3

На блок-схеме показаны ввод списка `numbers` и целевого числа `total`, инициализация списка позиций `indexes` и начального индекса $i = 0$, цикл последовательного просмотра элементов, проверка совпадения текущего числа x с эталоном `total`, сохранение индекса i в список совпадений при выполнении условия, слияние линий потока со стрелкой в точке соединения и вывод списка индексов. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 3 представлен на рисунке 8.

```
numbers = list(map(int, input().split()))
total = int(input())
indexes = []
for index, number in enumerate(numbers):
    if number == total:
        indexes.append(index)
print(f'Индексы: {indexes}')
```

Рисунок 8 – Текст программы для задания 3

Использование цикла `for index, number in enumerate(numbers)` позволяет одновременно получать порядковый номер и значение текущего элемента списка без необходимости ручного управления счетчиком.

Результат работы программы при вводе списка «4 7 4 4 9» и искомого числа 4 представлен на рисунке 9.

```
4 7 4 4 9
4
Индексы: [0, 2, 3]
```

Рисунок 9 – Результат работы программы для задания 3

1.4 Задание 4. Суммы строк и столбцов

В четвертом задании требуется вычислить суммы элементов каждой строки и каждого столбца для двумерной таблицы целых чисел. Пользователь задает размеры таблицы r (число строк) и c (число столбцов), после чего вводит r строк, каждая из которых содержит c целых чисел. Таблица сохраняется в памяти в виде списка списков. Накопление сумм по строкам и столбцам выполняется в циклах. Результаты выводятся в виде двух списков, порядок элементов в которых соответствует исходному расположению строк и столбцов.

Алгоритм решения четвертого задания представлен на рисунке 10.

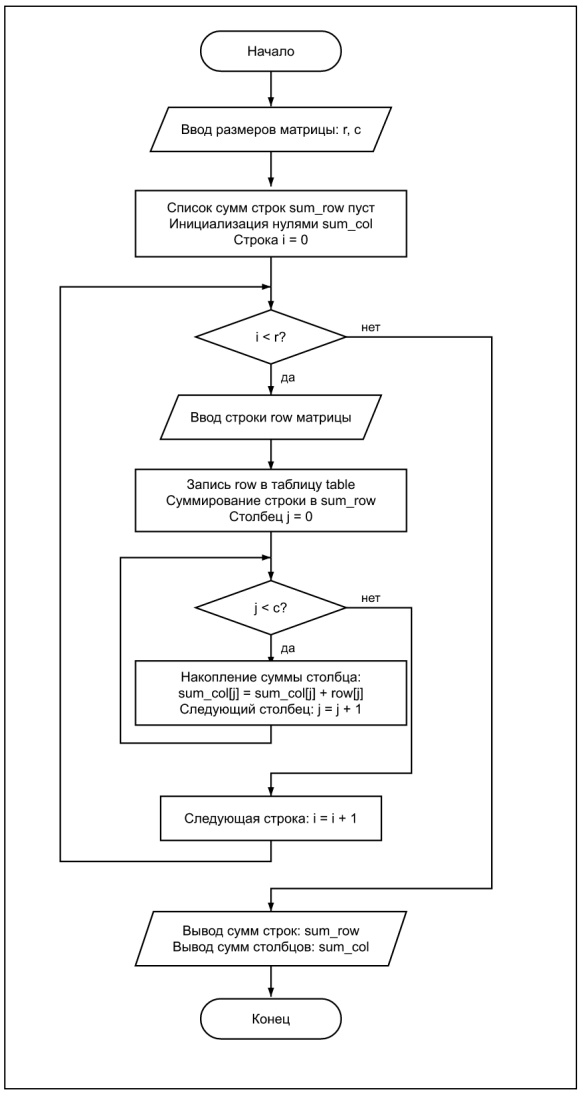


Рисунок 10 – Блок-схема программы для задания 4

На блок-схеме отражены ввод размерностей таблицы r и c, создание структур для хранения таблицы `table`, сумм строк `sum_row` и списка столбцовых аккумуляторов `sum_col` размера c, организация внешнего цикла по строкам $i < r$ с построчным вводом и суммированием элементов строки, организация внутреннего цикла по столбцам $j < c$ для накопления элементов `sum_col[j] = sum_col[j] + row[j]`, переход к следующей строке и вывод итоговых списков сумм. Блок-схема оформлена по ГОСТ 19.701-90.

Текст программы для задания 4 представлен на рисунке 11.

```
r, c = [int(part) for part in input().split()]
table = []
sum_row = []
sum_col = [0] * c
for i in range(r):
    row = [int(part) for part in input().split()]
    table.append(row)
    sum_row.append(sum(row))
    for j in range(c):
        sum_col[j] += row[j]
print(f'Суммы строк: {sum_row}')
print(f'Суммы столбцов: {sum_col}')
```

Рисунок 11 – Текст программы для задания 4

Список столбцовых сумм инициализируется как `[0] * c`, что обеспечивает прямой доступ к накапливаемому значению по индексу каждого столбца `sum_col[j] += row[j]` при чтении текущей строки.

Результат работы программы для матрицы размером 2 на 3 представлен на рисунке 12.

```
2 3
3 -1 4
0 5 2
Суммы строк: [6, 7]
Суммы столбцов: [3, 4, 6]
```

Рисунок 12 – Результат работы программы для задания 4

1.5 Задание 5. Поиск в области таблицы

В пятом задании требуется найти все ячейки прямоугольной подматрицы, содержащие заданное пользователем значение `target`. Пользователь вводит размеры матрицы r и c, строки таблицы, границы прямоугольной области (`row_start`, `row_end`, `column_start`, `column_end` включительно) и искомое целое число. Обход области осуществляется построчно сверху вниз и слева направо внутри каждой строки. Для каждого найденного совпадения выводятся индексы строки и столбца через пробел. При отсутствии вхождений выводится сообщение «Не найдено».

Алгоритм решения пятого задания представлен на рисунке 13.

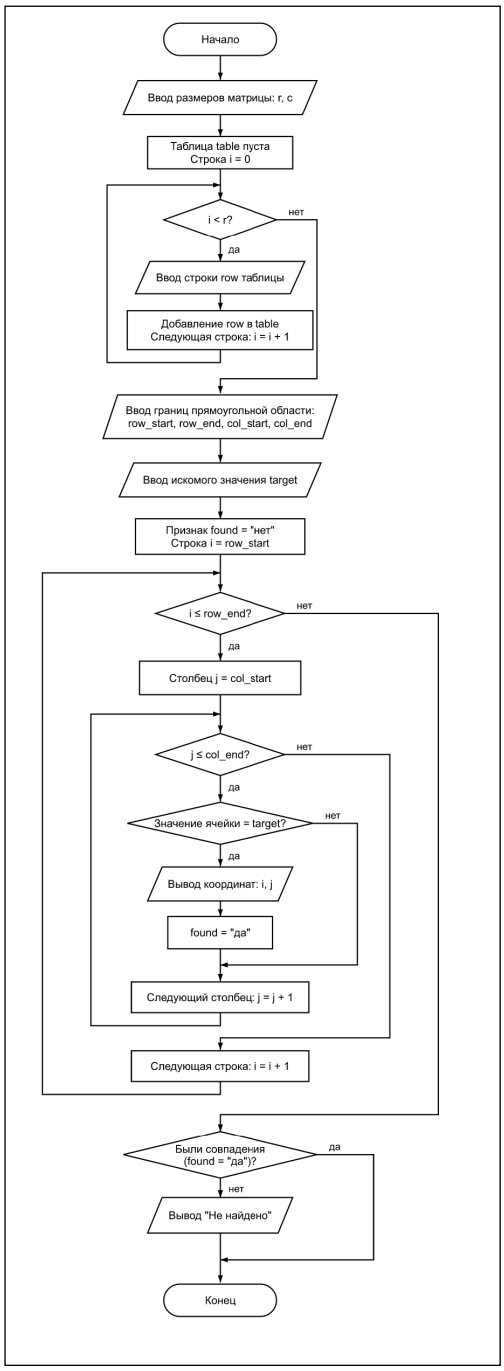


Рисунок 13 – Блок-схема программы для задания 5

На блок-схеме показаны ввод таблицы и параметров поиска (границы прямоугольной области и искомое число `target`), флаг обнаружения совпадений `found = "нет"`, вложенные циклы обхода строк от `row_start` до `row_end` и столбцов от `col_start` до `col_end`, проверка равенства значения ячейки числу `target`, вывод координат совпадения и фиксация флага `found = "да"`, инкремент индексов, а также финальная проверка наличия совпадений с выводом сообщения «Не найдено» при их отсутствии. Слияние обходной ветви с основной вертикальной линией перед блоком «Конец» выполнено со стрелкой в точке соединения в строгом соответствии с ГОСТ 19.701-90.

Текст программы для задания 5 представлен на рисунке 14.

```
r, c = [int(part) for part in input().split()]
table = []
for i in range(r):
    row = [int(part) for part in input().split()]
    table.append(row)
row_start, row_end, column_start, column_end = map(int, input().split())
target = int(input())
found = False
for i in range(row_start, row_end + 1):
    for j in range(column_start, column_end + 1):
        if table[i][j] == target:
            print(i, j)
            found = True
if not found:
    print('Не найдено')
```

Рисунок 14 – Текст программы для задания 5

Для обхода заданной прямоугольной области используются вложенные циклы `range(row_start, row_end + 1)` и `range(column_start, column_end + 1)` со смещением верхней границы на единицу для включения крайних элементов. Булева переменная `found` отслеживает факт нахождения хотя бы одного совпадения.

Результат работы программы для матрицы 3 на 4 при поиске значения 5 представлен на рисунке 15.

```
3 4
5 1 5 0
2 5 3 5
5 4 5 6
0 2 1 3
5
0 2
1 1
1 3
2 2
```

Рисунок 15 – Результат работы программы для задания 5